**CSCE 411: Design and Analysis of Algorithms**

# Week 8: Randomized Algorithms; Hashing; PageRank

*Date: March 24, 2026* *Nate Veldt, updated by Samson Zhou*

**Course Logistics**

- Randomized algorithms, hashing, and PageRank: Chapters 5, 11, and 20
- Homework 5 out, due this Friday

## 1 Randomized Algorithms: Motivating Problems

Randomized algorithms are used in several applications for analyzing large datasets and strings. We will now take a closer look at these applications.

**Probability reminders**

## 1.1 Monte Carlo and Las Vegas Algorithms

**Monte Carlo Algorithms.** A randomized algorithm whose runtime is deterministic, but has a small probability of _____.

**Las Vegas Algorithms.** A randomized algorithm that _____ but whose runtime is a random variable.

**One-sided Error** If a Monte Carlo algorithm can err on only one type of output (e.g., false positives but no false negatives), it has _____

**Probability Amplification** If we run a Monte Carlo algorithm with a bounded error probability multiple times, we can reduce the probability of failure to _____

**Question 1.** *Suppose a Monte Carlo algorithm for a decision problem has a probability of* $1/3$ *of returning an incorrect answer. If we run it* $k$ *independent times and take the majority vote, what happens to the probability of success?*

A *It remains* $2/3$

B *It approaches* $1$ *as* $k$ *increases*

C *It approaches* $0$ *as* $k$ *increases*

D *It becomes exactly* $1$ *after* $k = 3$ *iterations*

## 1.2 Fingerprinting and String Matching

The *string matching problem* asks us to find occurrences of a pattern $P$ of length $m$ in a text $T$ of length $n$. A naive algorithm takes time _____.

Instead of comparing the characters directly, we can compare the _____

**Rolling Hashes**

### 1.3 The Karp-Rabin Algorithm

A rolling hash of a string allows us to slide a window of length $m$ over the text $T$ so that:

## 2 Application 1: Checking if Two Strings Match

**Theorem 2.1.** *Fingerprints do not match* $\implies$ *strings are different. Equivalently:*

---

*Proof* First, ( $\implies$ ) we show that if the strings are different, the fingerprints can still match (a false positive or collision).

Next ($\Leftarrow$) we show that if strings are identical, their fingerprints will perfectly match.

## 3 Application 2: Hash Tables and Dictionaries

Given a universe of keys $U$, a dictionary maintains a dynamic set of elements supporting insert, delete, and search operations.

We can use the following procedure to resolve collisions using chaining:

1.

2.

**Theorem 3.1.** *For any deterministic hash function, an adversary can choose $n$ keys that all hash to the same slot, causing search times to be $\Theta(n)$.*

*Proof.* 1. Let $U$ be a universe of keys where $|U| > n \cdot m$

2. Our goal is to show that

   _____

3. When keys are mapped to $m$ slots, there are three different possibilities for the adversary:

   - **Case 1**: By the Pigeonhole Principle, at least one slot receives $\geq \lceil |U|/m \rceil$ keys.

     Thus, since $|U| > n \cdot m$, this slot gets $> n$ keys. Reason: _____

   - **Case 2**: The adversary selects $n$ keys from this heavily loaded slot.

     Reason: _____

   - **Case 3**: All $n$ keys are inserted into the hash table, forming a single linked list of length $n$.

     Searching for the last element takes time proportional to $n$. Reason: _____

4. In all cases that are possible, _____

□

## 4 Universal Hashing and Expected Chain Lengths

If $U$ is a universe of keys and $m$ is the table size, a *universal hash family* $\mathcal{H}$ is a collection of hash functions in which the probability of a collision between any two distinct keys

is at most $1/m$.

Let $x, y \in U$ be distinct keys and assume that $h$ is chosen uniformly at random from $\mathcal{H}$. The *expected chain length* for a hash table is bounded as follows:

- There is a random choice of $h \in \mathcal{H}$ at runtime
- There is a collision $h(x) = h(y)$ with probability $\leq 1/m$

**Lemma 4.1.** *The expected chain length for any key $x$ is* _____

The *load factor* of a hash table with $n$ elements and $m$ slots is $\alpha$.

$$\alpha = \frac{n}{m}$$

**Lemma 4.2.** *If we use universal hashing, the expected time for a search operation is*

_____

## 5 Introduction: Web Search and PageRank

Before Google, web search engines often struggled to rank results effectively, heavily relying on simple text matching. The **PageRank algorithm**, famously invented by Larry Page and Sergei Brin (founders of Google), assigns a *PageRank* (a score, or a measure of importance) to each webpage.

Given webpages numbered $1, \ldots, n$. The PageRank of webpage $i$ is based on its **linking webpages** (webpages $j$ that link to $i$), but we don't just count the number of linking webpages, i.e., we don't want to treat all linking webpages equally.

Instead, we **weight** the links from different webpages:

- Webpages that link to $i$, and have high PageRank scores themselves, should be given _____.
- Webpages that link to $i$, but link to a lot of other webpages in general, should be given _____.

Note that the first idea is circular! (But that's OK).

# 6 BrokenRank (almost PageRank) definition

Let $G = (V, E)$ be a directed graph representing the web, where $V$ is the set of web pages and $E$ is the set of hyperlinks.
Let $L_{ij} = 1$ if webpage $j$ links to webpage $i$ (written $j \to i$), and $L_{ij} = 0$ otherwise. Also let $m_j = \sum_{k=1}^{n} L_{kj}$, the total number of webpages that $j$ links to (also known as the out-degree $d_{out}(j)$).
First we define something that's almost PageRank, but not quite, because it's broken. The **BrokenRank** $p_i$ of webpage $i$ is:

$$p_i = \sum_{j \to i} \frac{p_j}{m_j} = \sum_{j=1}^{n} \frac{L_{ij}}{m_j} p_j$$

Does this match our ideas from the last section? Yes: for $j \to i$, the weight is $p_j/m_j$—this increases with $p_j$, but decreases with $m_j$.

## 6.1 BrokenRank as a Markov chain

Think of a **Markov Chain** as a random process that moves between states numbered $1, \ldots n$ (each step of the process is one move). Recall that for a Markov chain to have an $n \times n$ transition matrix $P$, this means $P(\text{go from } i \text{ to } j) = P_{ij}$.
Suppose $p^{(0)}$ is an $n$-dimensional vector giving initial probabilities. After one step, $p^{(1)} = P^T p^{(0)}$ gives probabilities of being in each state.
Now consider a Markov chain, with the states as webpages, and with transition matrix $A^T$. Note that $(A^T)_{ij} = A_{ji} = L_{ji}/m_i$, so we can describe the chain as:

$$P(\text{go from } i \text{ to } j) = \begin{cases} 1/m_i & \text{if } i \to j \\ 0 & \text{otherwise} \end{cases}$$

This is like a *random surfer*, i.e., a person surfing the web by clicking on links uniformly at random. A random surfer is a model of user behavior where a user on page $u$ follows an outgoing link to page $v$ with probability _____.
GenericRandomWalk($G$)

1. Set current node $u$ to a random page
2. While the surfer keeps browsing
3. find an outgoing edge $(u, v)$ uniformly at random
4. $u \leftarrow v$

## 6.2 Why is BrokenRank broken?

There's a problem here. Our Markov chain, a random surfer on the web graph, is **not strongly connected**, in three cases (at least):

- **Disconnected components**
- **Dangling links (Dead Ends).** Let $u$ be a node with no outgoing edges. If the random surfer reaches $u$, they get stuck, causing the probabilities to _____
- **Loops (Spider Traps).** Let $S$ be a set of nodes. If $S \subseteq V$ has no outgoing edges to $V - S$, we call it a _____. A random surfer _____ inside $S$ if _____.

Actually, even for Markov chains that are not strongly connected, a stationary distribution always exists, but may be **nonunique**. In other words, the BrokenRank vector $p$ exists but is **ambiguously defined** (yielding totally opposite rankings depending on start conditions).

## 7 PageRank as a Markov chain

**What is the key to making this work?** In the random surfer model, we essentially transition through the network's Markov chain to find the stationary distribution. The key to implementing this method is *ensuring the surfer doesn't get permanently trapped.* Let $A = \frac{1-d}{n}E + dLM^{-1}$, and consider as before a Markov chain with transition matrix $A^T$. Here $d$ is the *teleportation parameter* (often set to 0.85).
Well $(A^T)_{ij} = A_{ji} = (1-d)/n + dL_{ji}/m_i$, so the chain can be described as:

$$P(\text{go from } i \text{ to } j) = \begin{cases} (1-d)/n + d/m_i & \text{if } i \to j \\ (1-d)/n & \text{otherwise} \end{cases}$$

The chain moves through a link with probability $(1-d)/n + d/m_i$, and with probability $(1-d)/n$ it jumps to an unlinked webpage.
Hence this is like a **random surfer with random jumps**. Fortunately, the random jumps get rid of our problems: a teleportation jump _____ the trap by randomly jumping anywhere in the graph. Our Markov chain is now **strongly connected**. Therefore the stationary distribution (i.e., PageRank vector) $p$ is **unique**.
A *PageRank vector* of $G$ is a probability distribution $PR$ that satisfies over all nodes in

the network. A PageRank vector is a _____ if it remains unchanged after an update step.

## 7.1 Generic teleportation strategy

We define the following *teleportation* strategy for updating a node's PageRank score: GenericPageRankUpdate$(G, v, d)$

1. Let $jump = (1 - d)/|V|$ be the probability of teleporting to $v$
2. Let $link_sum = \sum_{(u,v) \in E} d \cdot \frac{PR(u)}{d_{out}(u)}$ be the probability of arriving via links
3. Return $jump + link_sum$

**Lemma 7.1.** *If $d$ is a damping factor between 0 and 1, then* GenericPageRankUpdate *will ensure the Markov chain is irreducible and aperiodic.*

## 7.2 Random Walk Strategies

The simple random walk and the teleportation random walk are two strategies for traversing $G = (V, E)$.

| **Strategy** | **What is the state transition?** | At each step we transition: |
|---|---|---|
| Simple Random Walk | | |
| Teleportation Walk | | |

# 8 Computing PageRank

We can compute $p$ by repeatedly multiplying by $A$. The following algorithm will compute the PageRank of nodes in a web graph $G = (V, E)$:
PageRank-Power-Iteration(G)

1. Initialize $PR(u) = 1/|V|$ for each $u \in V$.
2. Identify the *teleportation parameter* $d$ (often set to 0.85).
3. Update the PageRank of each node by summing the contributions from its incoming edges, and applying the teleportation jump.
4. Output the converged PageRank scores after the values stabilize.

## 8.1 Power Iteration Algorithm in More Depth

During the Power Iteration algorithm, we update the scores over a series of iterations. Each node has the following attributes:

- $u$.score = current PageRank score of $u$
- $u$.out_degree = number of outgoing edges from $u$

We will use a secondary array new_score to store the updated probabilities during the iteration, which allows us to perform synchronous updates in $O(V + E)$ time.

```
PageRank-Power-Iteration(G, d)
  for u ∈ V do
      u.score = 1/|V|
  end for
  converged = FALSE
  while converged == FALSE do
      converged = TRUE
      for v ∈ V do
          new_score[v] = (1 − d)/|V|
          for u ∈ InEdges[v] do
              new_score[v] = new_score[v] + d u.score / u.out_degree
          end for
          if |new_score[v] − v.score| > ϵ then
              converged = FALSE
          end if
      end for
      for v ∈ V do
          v.score = new_score[v]
      end for
  end while
  Return scores
```

## 9 A Basic Web Search

For a basic web search, given a query, we could do the following:

1. Compute the PageRank vector $p$ **once** (Google recomputes this from time to time, to stay current)
2. Find the documents containing all words in the query
3. **Sort** these documents **by PageRank**, and return the top $k$ (e.g., $k = 50$)

**Question 2.** *What is the runtime of one full iteration of the PageRank algorithm, knowing that it requires visiting every node and traversing every incoming edge?*

A $\Theta(\log V)$

B $\Theta(\log E)$

C $\Theta(V + E)$

D $\Theta(E \log V)$

E $\Theta(V^2)$